Task 1: Execute all the templates shown in the video.

There are 3 types

1st type :- creating a variable.tf file and keeping this variable name in main.tf is away of dynamic calling

```
resource "local_file" "my_pet" {
  filename = var.filenmae
  content  = var.content
}
resource "random_pet" "sunith" {
  prefix    = "chintu"
  separator = "."
  length    = 1
}
```

2nd type-:

i) Runtime calling: Creating variable.tf file and keeping it empty with

```
variable filename{

}
variable content{

}
```

ii) Runtime calling: -:

```
terraform apply -var "filename=sunith.txt" -var "content=i just believe in my self"
```

Life cycle:

In Lifecycle we have 3 types

→ create before destroy
→ignore changes
→ prevent destroy

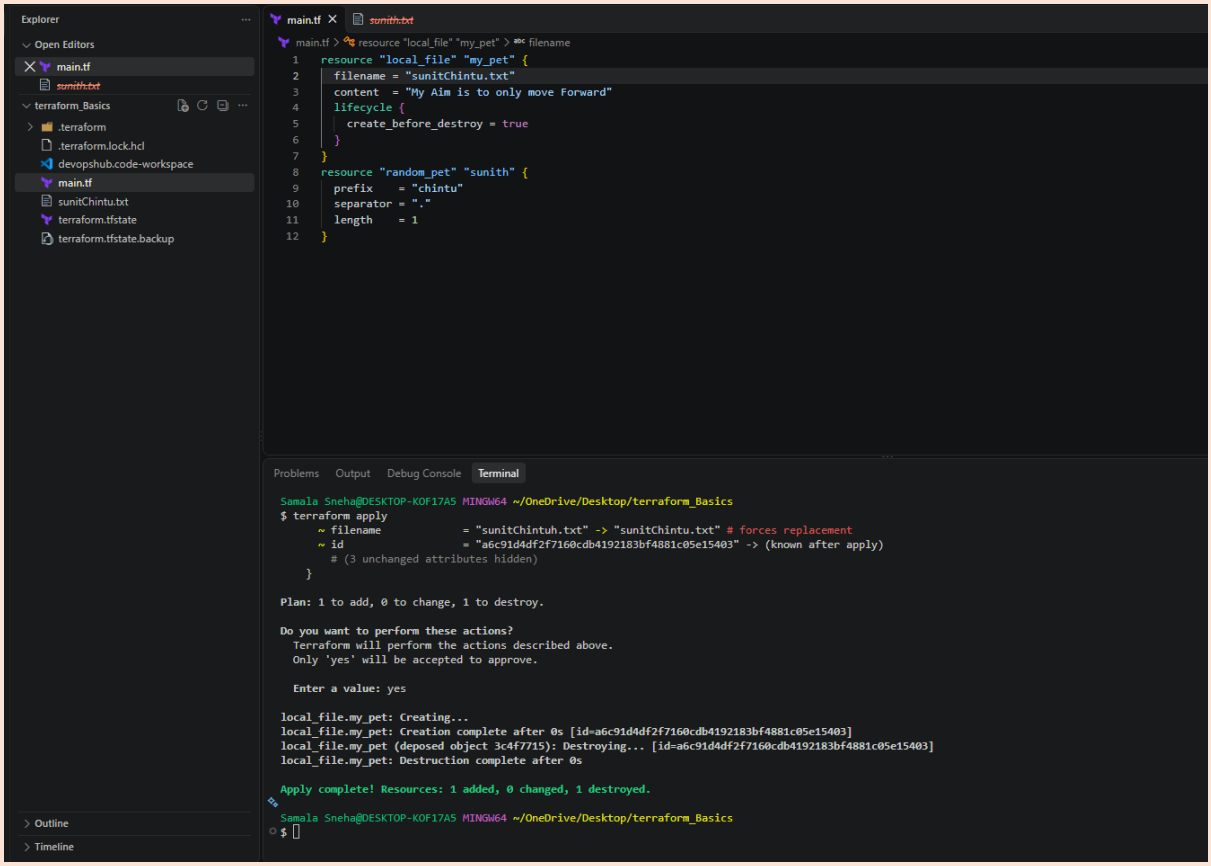
create before destroy

Here in this below screen shot we can see after adding

```
lifecycle {
Create_after_destroy = true”
}
```

Post running the output it has changed to creating then destroy as highlighted and the output has been updated in statefile

Implementation:



ignore_changes

Here in this below screen shot we can see after adding

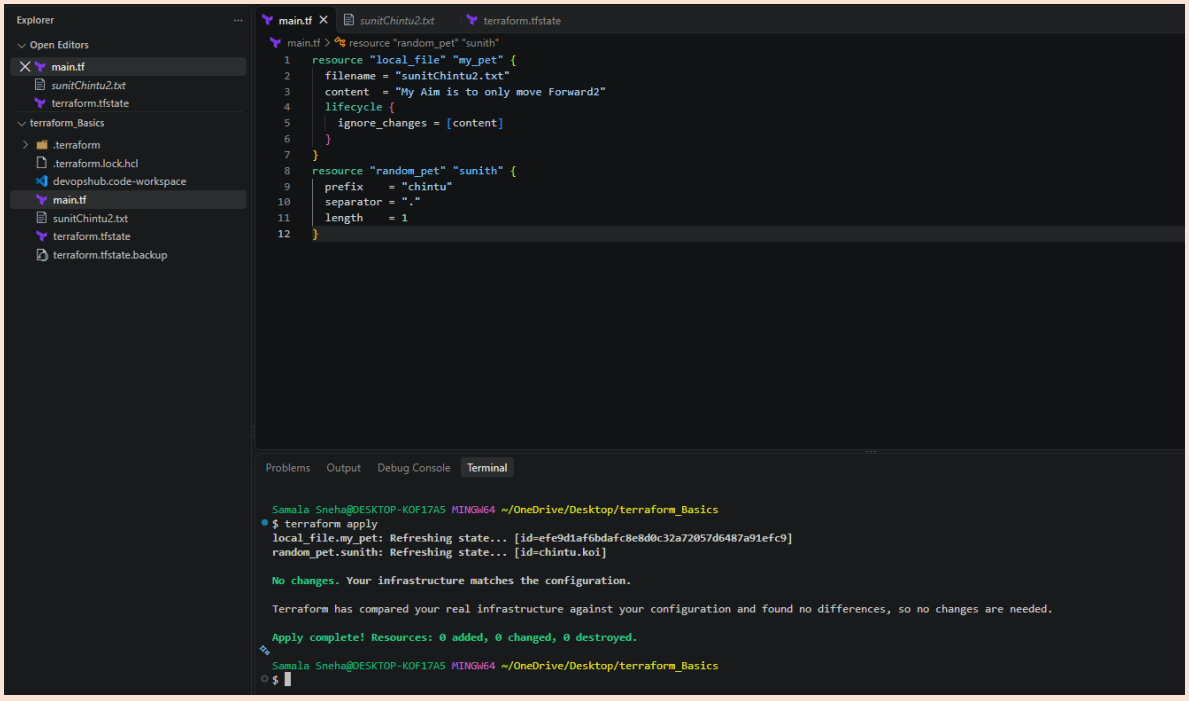
lifecycle {

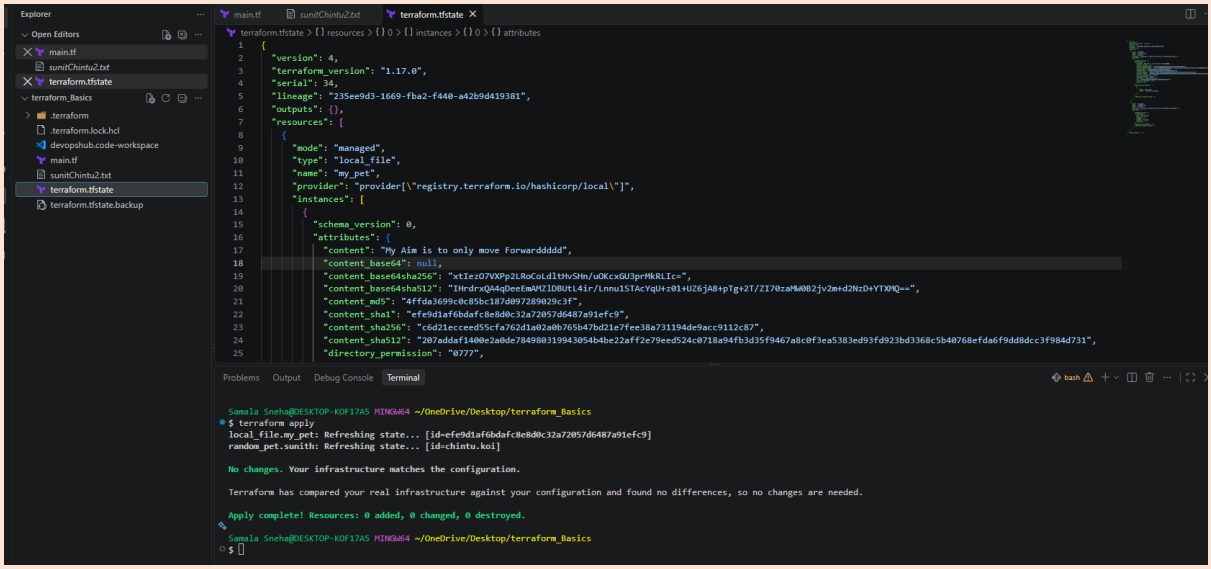
ignore_changes=[

content]

}

Implementation:



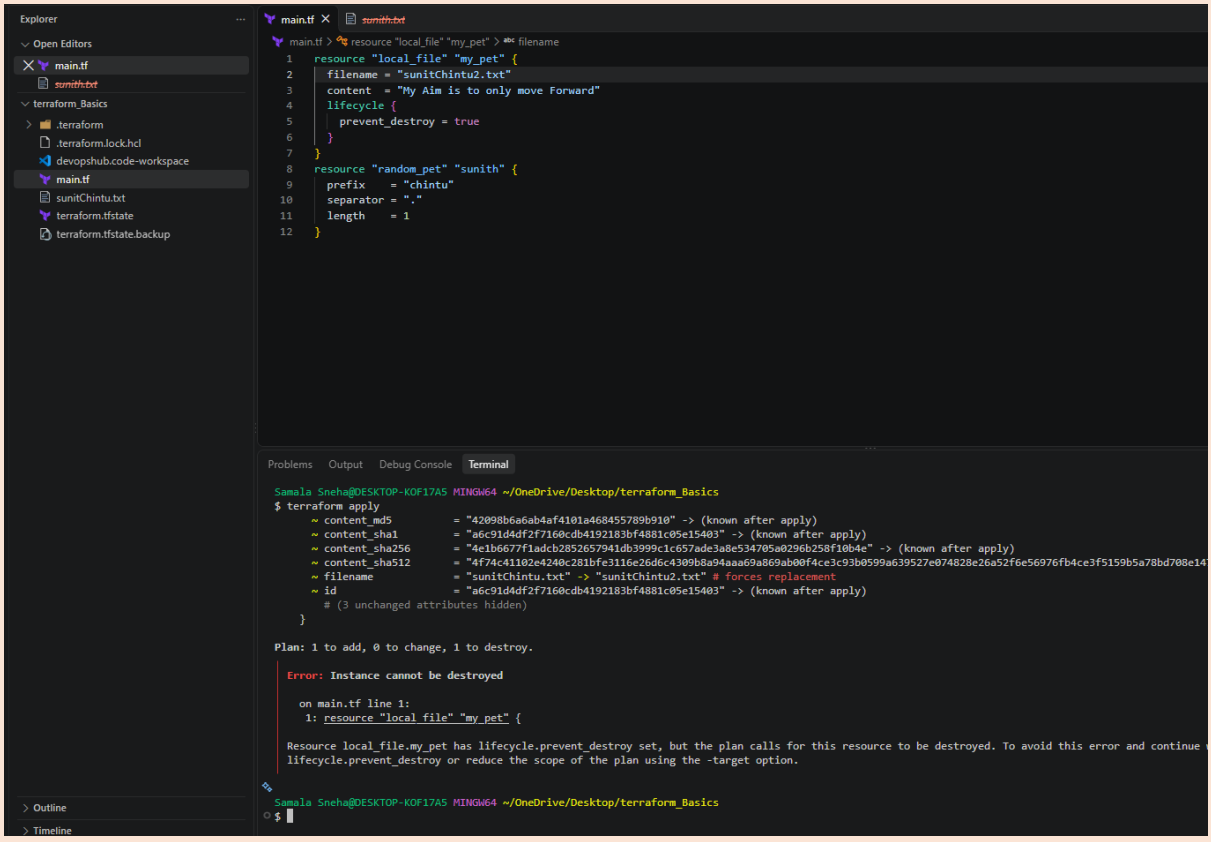


Post running the output the value won't get updated in tfstate file despite being changed in cli because of
ignore_changes=[content]

prevent destroy
Here in this below screen shot we can see after adding

```
lifecycle {
prevent_destroy = true
}
```

Implementation:



This will stop destroying the content of file because of prevent_destroy = true
If we still want to delete the file then we can do the terraform destroy

Task 2: Integrate Terraform in Jenkins Using Terraform Plugin

1. Objective

To integrate Terraform with Jenkins using a Declarative Pipeline and automate the creation of resources using Terraform configuration.

2. Purpose

The purpose of this task is to understand how Jenkins Pipeline can execute Terraform commands to initialize providers, create an execution plan, apply the configuration, and verify the generated resources.

In this lab, Terraform creates a local text file and generates a random pet name using the local and random providers. No AWS infrastructure is created.

3. Implemented Clear Steps

Step 1: Install and Verify Terraform

Terraform was installed on the Amazon EC2 instance.

Command: terraform -version

Output:
Terraform v1.17.0
on linux_amd64

Terraform executable path: /usr/bin/terraform

The Jenkins service user was also verified to have permission to execute Terraform:
sudo -u jenkins /usr/bin/terraform -version

Step 2: Configure Terraform in Jenkins

1. Log in to the Jenkins dashboard.
2. Navigate to **Manage Jenkins → Tools**.
3. Locate **Terraform installations**.
4. Configure the following values:
    - Name: Terraform-Local
    - Install directory: /usr/bin
    - Install automatically: Unchecked
5. Click **Save**.

This configuration identifies the existing Terraform installation for Jenkins.

Step 3: Create a Jenkins Pipeline Job

1. Open the Jenkins Dashboard.
2. Click **New Item**.
3. Enter the job name: terraform-pipeline-lab

4. Select **Pipeline**.
5. Click **OK**.
6. Scroll to the **Pipeline** section.
7. Set **Definition** to Pipeline script.
8. Paste the following Declarative Pipeline script.

Step 4: Configure the Pipeline Script

```
pipeline {
    agent any

    stages {
        stage('Create Terraform Files') {
            steps {
                writeFile file: 'main.tf', text: '''
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
    random = {
      source = "hashicorp/random"
    }
  }
}

resource "local_file" "my_pet" {
  filename = "sunitChintu2.txt"
  content  = "My Aim is to only move Forward2"
}

resource "random_pet" "sunith" {
  prefix    = "chintu"
  separator = "."
  length    = 1
}
'''
            }
        }

        stage('Terraform Init') {
            steps {
                sh '/usr/bin/terraform init'
            }
        }

        stage('Terraform Plan') {
            steps {
                sh '/usr/bin/terraform plan'
```

```
        }
      }

      stage('Terraform Apply') {
        steps {
          sh '/usr/bin/terraform apply -auto-approve'
        }
      }

      stage('Verify Output') {
        steps {
          sh '''
            cat sunitChintu2.txt
            /usr/bin/terraform output || true
            ls -l sunitChintu2.txt
          '''
        }
      }
    }
}
```

Click **Save**.

Step 5: Execute the Pipeline

1. Open the terraform-pipeline-lab job.
2. Click **Build Now**.
3. Click the generated build number.
4. Open **Console Output**.
5. Verify that the stages execute successfully and the build finishes with:

Finished: SUCCESS

The Pipeline performs these operations:

- **Create Terraform Files:** Creates main.tf inside the Jenkins workspace.
- **Terraform Init:** Downloads and initializes the required providers.
- **Terraform Plan:** Checks the changes Terraform intends to make.
- **Terraform Apply:** Creates the resources described in the configuration.
- **Verify Output:** Displays the generated text file and checks that it exists.

Step 6: Verify the Generated File

The Jenkins workspace is: /var/lib/jenkins/workspace/terraform-pipeline-lab/

Run the following command on the EC2 instance:
sudo cat /var/lib/jenkins/workspace/terraform-pipeline-lab/sunitChintu2.txt

Expected output: My Aim is to only move Forward2

The file was successfully created, and its contents matched the Terraform configuration.

The workspace also contains:

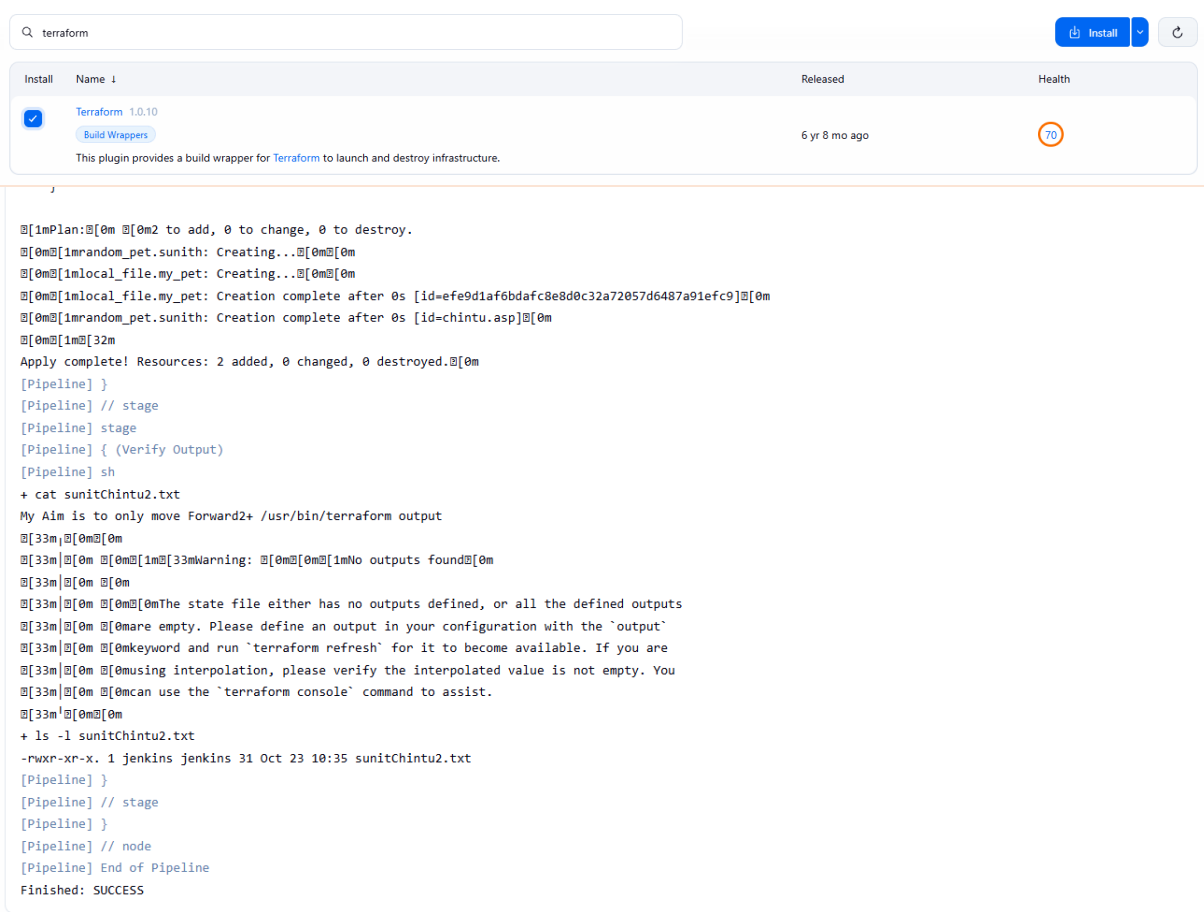
- main.tf — Terraform configuration file.

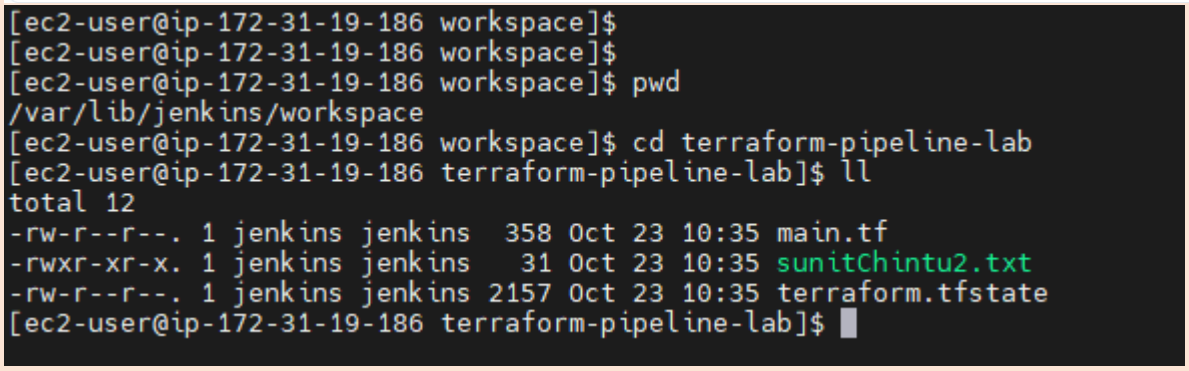
- sunitChintu2.txt — File created by the local_file resource.
- terraform.tfstate — Terraform state file tracking the managed resources.

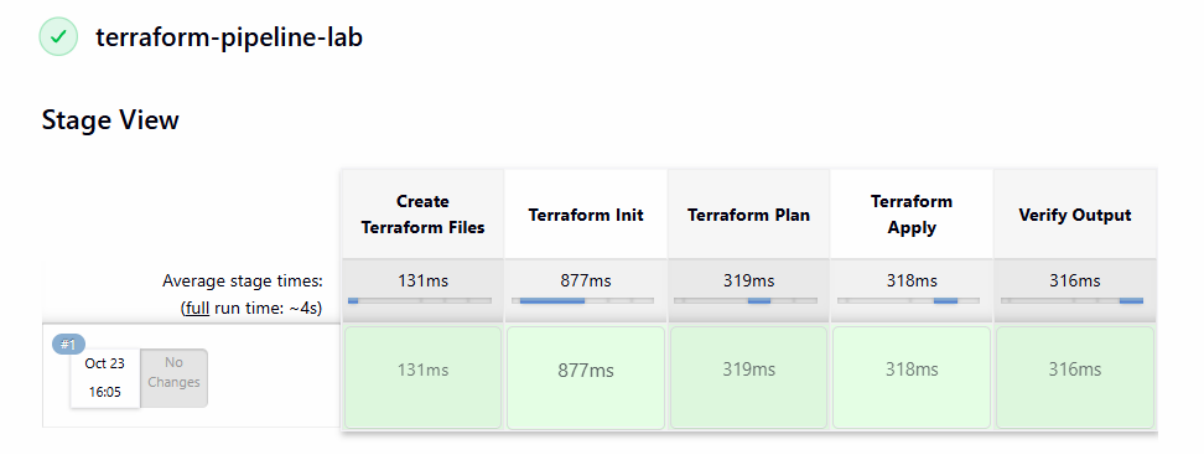
The random_pet.sunith resource generates a name with the chintu prefix and a period separator. Its generated name can be checked in the Terraform state or with an appropriate Terraform output.

Important: The terraform.tfstate file contains resource state information. Do not publish it to a public repository.

Implementation:







4. Conclusion

Terraform was successfully integrated into a Jenkins Declarative Pipeline. The Pipeline initialized the required providers, generated a plan, applied the Terraform configuration, and created sunitChintu2.txt with the expected content.

This lab demonstrated how Jenkins Pipeline automates Terraform execution and verifies the result without creating AWS infrastructure.